

What this welfare platform actually is, explained like you've never seen the spec

A walk-through of WELF-PRD-001 v2.1 for anyone who wants to understand the project without reading 24 sections of requirements language. No background assumed.

COMPANION TO THE FULL SPEC

IN THIS GUIDE

1. The one-paragraph version
2. The actual problem, in human terms
3. Why six AI teams, and what their "votes" mean
4. One app, many tenants, invisible walls
5. Rules are data, not code
6. Why we're tracking money like an accountant
7. Who's allowed to do what
8. Stopping people from gaming the system
9. What the AI is (and isn't) allowed to do
10. Keeping the shop's rent separate from the till
11. What actually happens when you join a group
12. Belonging to more than one group at once

13. Voting on more than just who's in charge
14. A wallet you can top up, but not empty out
15. Paying for someone else
16. Two signatures before the group's money moves
17. What gets built first
18. Decisions that have now been made
19. What's still an open question
20. A short glossary

THE BIG IDEA

The one-paragraph version

Thousands of small associations — staff welfare groups, alumni unions, church groups, civil-service welfare schemes — all do roughly the same thing: members pay dues, and the group pays out money when something happens (a birth, a death, a wedding, an illness). We want one mobile app that any of them can download, set up themselves, and run — without a programmer ever having to touch the code again just because a new group's rules are different.

That last part is the whole trick. Every one of these groups has its own "constitution" — a handwritten or typed rulebook saying how much people pay, when they qualify for help, and how much they get. If we build the app around one group's rulebook, it breaks the moment a second group with different rules signs up. So the entire platform is built around one core discipline: **every number, every waiting period, every payout amount lives in a settings screen, not in the program's source code.**

WHY THIS IS GENUINELY HARD

The actual problem, in human terms

To make sure we weren't just guessing at what "different rules" means, we looked at four real rulebooks from four very different organisations:

- A **university's** staff welfare association, spread across four campuses, with a Board of Trustees, a Management Team, and two committees.
- A **private school's** staff association, small and single-site, run by five elected officers.
- A **government ministry's** welfare scheme, with formal annual audits and a strict paper trail.
- An **alumni association** of a graduating class, run entirely by volunteers with no employer involved at all.

Every single one of them pays a different amount, on a different schedule, into a different set of benefits, with different waiting periods and different penalties for falling behind. Here's a small taste, translated out of formal language:

ONE GROUP'S RULE

Pay GHS 15 every month, straight out of your salary.

ANOTHER GROUP'S RULE

Pay GHS 100 once a year, plus GHS 20–50 whenever something happens to someone.

ONE GROUP'S RULE

Wait 6 months after joining before you can claim anything.

ANOTHER GROUP'S RULE

Wait 3 months, and if you ever leave and come back, you serve the waiting period all over again — plus an apology if you left on bad terms.

And it's not just numbers. One group pays a benefit to *a member's family* when the member dies. Another treats a member's own death as something the member's estate is compensated for directly. Those aren't the same rule with a different price tag — they're structurally different ideas about who gets paid. A rulebook that hardcodes "death benefit goes to the family" would simply be wrong for the second group.

THINK OF IT LIKE...

...building one tax-filing app for every country in the world, instead of one country. You wouldn't hardcode "income tax is 20%" into the software — you'd build a settings screen where each country's tax authority fills in its own rates, brackets, and deadlines. This platform is that same idea, applied to welfare-fund rulebooks instead of tax law.

HOW WE STRESS-TESTED OUR OWN THINKING

Why six AI teams, and what their "votes" mean

Rather than trust one pass at this design, the same brief and the same four rulebooks were handed to six different AI systems, working independently — none of them could see what the others were doing. Then all six write-ups were compared side by side.

The logic is simple: if six independently-working teams all land on the same answer without talking to each other, that's a strong sign the answer is actually right, not just a guess one of them happened to make. If they land on *different* answers, that tells us the question is genuinely open, and a human needs to make the call rather than have it slip through unnoticed.

EVERYONE AGREED ON THIS

All six independently said: keep every group's data in shared infrastructure, but wall it off so strictly that one group can never see another's members or money, no matter what.

WHAT THAT MEANS FOR YOU

This is now treated as settled — no further debate needed. It's in the full spec as a "high confidence" decision.

EVERYONE AGREED SOMETHING WAS NEEDED, BUT DISAGREED ON HOW

All six agreed you need a way to stop people from ignoring dues for months and then suddenly paying everything back right before they want a payout — but each proposed a different specific mechanism.

WHAT THAT MEANS FOR YOU

We combined all of their ideas into one layered system rather than picking a winner (Section 8 of this guide).

Two decisions actually got *stronger* once we compared notes: keeping proper double-entry books (like a real accountant, not a checklist) turned out to have three of the six independently insisting on it, not just one. And keeping the app's own subscription fees completely separate from members' welfare money turned out to be something two totally separate teams stated in almost identical words, unprompted.

THE FOUNDATION

One app, many tenants, invisible walls

THINK OF IT LIKE...

...an apartment building. Every organisation gets its own "apartment" — its own members, its own money, its own rules. They all live in the same building (the same app, the same servers), which keeps the landlord's costs down and means a new tenant can move in without anyone building them a new house from scratch. But no tenant can see into another tenant's apartment, and there's no master key that accidentally opens two doors at once.

In the spec's language, this is called **multi-tenancy**, and the specific approach — shared database, strict digital walls enforced by the database itself rather than just by the app's good behaviour — is the single most agreed-upon decision in the whole document. It also has an escape hatch: if one organisation grows huge (think: a university with tens of thousands of staff), it can be moved into its own private wing of the building without redesigning anything.

THE CORE TRICK

Rules are data, not code

This is the idea the whole project stands on, so it's worth spelling out slowly.

A lazy way to build this app would be to write, in the program itself: *"if this member's organisation is the university, charge GHS 15 a month."* That works for exactly one organisation. The moment a second one signs up with a different amount, someone has to open the

program's source code, add another line, test it, and re-release the app. Do that for a thousand organisations and you have a thousand special cases nobody can safely change ever again.

Instead, every number a group might want to change — the monthly due, the waiting period, the payout for a birth, the penalty for missing three payments — lives in a settings table the group's own administrator fills in through the app itself, the same way you'd fill in a form. The program that reads those settings and does the math is generic: it doesn't know or care whether it's calculating a university's birth benefit or an alumni group's wedding gift. It just reads whatever numbers that organisation put in its own settings and applies them.

THINK OF IT LIKE...

...a mail-merge template versus writing a thousand individual letters by hand. The template has blanks for name, amount, and date; you fill in different values for different people, and the same template produces a correctly personalised letter every time. Nobody rewrites the template just because a new person's name is different.

One wrinkle worth knowing about: organisations amend their own rulebooks over time — a monthly due goes from GHS 20 to GHS 30, say. The system has to remember *both* figures: the old one, for anything that happened before the change, and the new one, going forward. Otherwise a member who died last year under the old rules would suddenly, on paper, seem to have been owed money under rules that didn't exist yet when they died. Every rule, in other words, comes with a "since when" date attached, and old versions are never deleted — only replaced going forward.

HANDLING REAL MONEY RESPONSIBLY

Why we're tracking money like an accountant, not like a checklist

The tempting, simple way to track payments is a single table: *Member, Amount, Paid? (yes/no)*. It looks fine until something goes wrong — a payment gets refunded, a benefit gets reversed, an admin made a typo — and now you're editing history instead of correcting it, and nobody can reconstruct what the books actually said on any given day in the past.

Three of the six independent teams insisted on something closer to how a real accountant keeps books: **double-entry bookkeeping**. Every cedi that moves is written down twice — once as coming *from* somewhere, once as going to somewhere — and the two sides always have to balance. Once a record is written, it's never edited or deleted. If a mistake happens, you don't erase it; you add a new, clearly-labelled correcting entry that cancels it out, the same way an accountant would draw a line through an error rather than use correction fluid.

THE NAIVE VERSION

"Kofi paid GHS 20." One row. If it's wrong, someone edits the row.

THE ACCOUNTANT'S VERSION

"GHS 20 moved out of Kofi's obligation and into the Funeral Fund's cash account." Two sides, always balanced, never edited — only corrected with a new, linked entry.

This matters more than it sounds like it should, because welfare associations get audited — sometimes by their own members, sometimes by an external auditor within a strict deadline. An auditor's first question is usually "prove this," and a system that can only show the current total, with no trail of how it got there, can't prove anything.

One more distinction worth knowing: the system separates **"what was owed"** from **"what was actually paid."** If someone was supposed to pay GHS 50 in January and only pays it in June, the system needs to remember it was late — not quietly relabel it as an on-time January payment just because the money eventually arrived. That memory is what makes it possible to catch people who deliberately let their dues slide and then pay everything back right before they want to make a claim (more on that in the next section).

WHO CAN TOUCH WHAT

Who's allowed to do what

THINK OF IT LIKE...

...a nightclub with different wristbands. The bouncer at the door doesn't decide who gets which wristband on the fly — the club owner decided in advance who gets VIP access,

who can go backstage, and who's general admission only. Each welfare group is its own "club" and gets to hand out its own wristbands however its own constitution says to, rather than the app forcing every group to use the same fixed job titles.

In practice this means a group doesn't get a hardcoded list of roles like "Treasurer" and "Secretary" baked into the app. Instead, each organisation builds its own roles out of individual permissions — "can approve a claim," "can see financial reports," "can change the monthly due amount" — and decides for itself who gets which combination. A tiny alumni group might give one person almost everything; a government ministry's welfare scheme, which has committees and audit rules, will want much finer separation.

One safeguard worth calling out by name: several of the AI teams, and most of the real rulebooks, independently insisted that **the person who starts a financial action shouldn't also be the only person who approves it**. If a treasurer records a payment reversal, someone else — not the treasurer — should be the one who signs off on it. This is a very old idea in accounting (it's called "maker-checker" or "segregation of duties") and it's one of the few places the app can enforce good behaviour rather than just hoping people follow the rules.

THE FREE-RIDER PROBLEM

Stopping people from gaming the system

Every mutual-aid scheme has the same vulnerability: someone stops paying dues for months, has nothing to show for their membership — and then, the moment they actually need help (a funeral, a hospital bill), suddenly pays everything they owe in one lump sum and expects a payout as if they'd been a model member all along. The associations call this "dumping." It's the single biggest fairness risk in the whole system, because if it's not stopped, the honest members who paid every month are effectively subsidising the people who only show up when they need something.

The platform fights this with several layers working together, rather than one blunt rule:

- **A consistency score, not just a yes/no flag.** The system remembers a member's whole payment history — not just "are they currently paid up" but "how consistently have they

paid since joining" — so someone with a spotless record and one bad month looks very different from someone who's paid twice in three years.

- **A cooling-off period after catching up.** If someone clears months of missed payments all at once, any claim they file shortly afterward gets automatically flagged for a human committee to look at closely — not rejected outright, but not quietly approved either.
- **Losing the "first time" perks if you've already claimed before.** Several rulebooks give a better exit refund to someone who's *never* received a benefit than to someone who has — the system tracks that permanently, the way an insurance no-claims discount works in reverse.
- **Real deadlines for advance notice.** Some benefits (like a wedding gift) require the group to be told in advance — the system timestamps the request the moment it comes in, so nobody can quietly claim they gave two weeks' notice after the fact.

None of this is about being harsh — every one of the four real rulebooks we studied has some version of a grace period, because life happens and people fall behind for good reasons. The goal is just to make sure the system can tell the difference between "fell behind and is catching up honestly" and "only ever pays when there's a payout to collect."

WHERE ARTIFICIAL INTELLIGENCE FITS — AND WHERE IT DOESN'T

What the AI is (and isn't) allowed to do

There are two places AI genuinely helps here, and one hard line it's never allowed to cross.

- **Reading a new group's rulebook for them.** When a new organisation signs up, instead of an administrator manually typing every dollar amount and waiting period into a settings screen, they can upload their actual constitution document and the system will propose a first draft of the configuration — "it looks like your monthly due is GHS 20, your waiting period is 3 months, here's a draft benefit list." A human still has to look at every line and approve it before anything goes live. Nothing the AI reads out of a document becomes a real, active rule on its own.
- **Answering members' everyday questions.** "How much do I owe?" "Am I eligible for the funeral benefit?" "Why was my claim rejected?" — a member-facing assistant can answer

these honestly and specifically, because it's only allowed to look at that member's own real data and that organisation's actual current rules. It never guesses, and it never sees another member's information.

THE LINE THAT NEVER GETS CROSSED

No AI system in this platform is ever allowed to approve a claim, release a payment, change a financial rule, or alter anything in the ledger. It can draft, suggest, flag, and explain — but the actual decision to move money always has to come from a human being who's accountable for it. This isn't a technical limitation; it's a deliberate design rule, because the money involved is real and belongs to real people.

TWO POTS OF MONEY THAT MUST NEVER MIX

Keeping the shop's rent separate from the till

THINK OF IT LIKE...

...a landlord and a tenant's cash register. The company that runs this app is like a landlord: it might charge each welfare association a subscription fee to use the software, the same way Netflix charges a monthly fee. That subscription money belongs to the company running the platform. It has absolutely nothing to do with the welfare fund itself — the actual dues members pay in, and the payouts the association makes to its own members. Those are two completely separate pots of money, run by two completely different parties, and the app is built so they can never accidentally touch.

This sounds obvious, but it's easy to get wrong by accident if the software isn't designed carefully — for instance, if "money coming into the platform" were treated as one big undifferentiated stream. Two of the six independent AI teams flagged this exact risk, unprompted, using almost identical language, which is part of why it's treated as a firm requirement rather than a nice-to-have.

FROM NOTHING TO A FULL MEMBER

What actually happens when you join a group

Two slightly different stories, depending on whether this is your very first welfare group or your second.

The first time. Someone invites you — a link, a QR code, or your employer adding you from a staff list. You verify your phone number once, which creates your one login for the whole app. You fill in your details and, importantly, list who your dependants are (spouse, children, parents) *before* anything happens to them — because a bereavement payout later gets checked against names that were already on file, not names added the week of the funeral. The group's own waiting period then starts counting down before you're eligible for anything.

The second time — joining a different group. You already have a login, so there's no re-verifying your phone number — you just add a new membership to the same account, the same way described in the next section. The app offers to copy over the dependants you already listed for your first group, to save you retyping everything — but it makes you confirm each one again before it counts, because different groups have different ideas about who qualifies as a "recognised" spouse or child. Nothing is silently assumed to be true just because it was true somewhere else.

ONE PERSON, SEVERAL MEMBERSHIPS

Belonging to more than one group at once

In real life, people belong to several of these groups at the same time — the same person might be in their university's staff welfare scheme *and* their old school's alumni association *and* a hometown welfare group. Each of those has to stay completely separate: what you owe one group, and what you're eligible for there, must never leak into another.

THINK OF IT LIKE...

...one Google account that belongs to several different Workspaces — your personal Gmail, your employer's Workspace, a volunteer group's Workspace. It's still one login, but

each Workspace only sees what's inside it. Switching between them is one tap; nothing about one bleeds into another.

This platform works the same way: one phone number and login gets you into the app once, and then a simple switcher lets you move between whichever welfare groups you belong to. Whichever group is currently active is always shown clearly on screen, so you never accidentally check the wrong group's balance or pay dues into the wrong pot.

BEYOND ELECTING OFFICERS

Voting on more than just who's in charge

Every group already needed a way to elect its own Chairman, Treasurer, and Secretary. But groups also vote on things that aren't about a person at all — "should we raise the monthly due to GHS 30?", "should we approve this amended constitution?", "should we make an exception for this member's claim?" The platform treats these as the same underlying idea: a vote is a vote, whether the thing being decided is a candidate or a proposal.

One extra wrinkle worth knowing: whether the running tally is visible *while* voting is happening, or only revealed once voting closes, is left as a choice each group makes for itself — some committees want the transparency of watching the count live; others worry that seeing early results changes how people vote (the same reason many real elections don't announce results until polls close everywhere). The platform defaults to hiding the count until close, but any governance body can switch that on for its own votes.

A MIDDLE GROUND BETWEEN "PAY PER DUE" AND "CASH IN THE APP"

A wallet you can top up, but not empty out

Members can put money into a personal balance inside the app — a wallet — and then draw on it whenever a due or a levy comes up, instead of having to make a separate payment every single time. Think of it like topping up a store gift card: money goes in once, and gets used gradually.

ONE DELIBERATE LIMIT

You can put money in, and you can spend it on your own dues and levies. You **cannot** take it back out as cash, and you cannot send it to another member. That's not an oversight — it's the line that keeps this a tool for managing an association's own dues, rather than turning into something closer to a mobile-money account, which would drag the whole platform into a much heavier set of financial regulations (in Ghana, that's the Bank of Ghana's rules for companies that hold and move electronic money on people's behalf). Keeping the wallet one-directional — in, and only out via your own dues — keeps things simple and keeps the platform squarely a piece of software, not a financial institution.

WHEN YOU CAN'T PAY RIGHT NOW, BUT SOMEONE ELSE CAN

Paying for someone else

Sometimes the person who owes the money isn't the person who can pay it — a daughter covering her elderly father's monthly due, a friend helping out someone who's temporarily out of mobile money credit. The app lets anyone pay toward another member's due directly: pick the person, pay as usual, and the payment settles *their* obligation while the charge comes out of *your* phone or card.

WITHOUT THIS FEATURE

The person struggling to pay just... doesn't pay, and starts sliding toward defaulter status, even though someone nearby was willing and able to help.

WITH THIS FEATURE

The payment is recorded honestly on both sides — the payer's own history correctly shows it wasn't their own due, and the recipient's history shows "settled, paid by [name]" — nobody's records get confusing or misleading.

That honest record-keeping matters for another reason too: without it, the system might wrongly flag "the same phone number paying for several different people" as suspicious (Section 22.1 of the full spec lists exactly this as a fraud signal). Because a proxy payment always

names both people clearly, the system can tell the difference between a parent legitimately paying for their children and someone quietly running several unrelated accounts off one phone.

SPENDING THE GROUP'S OWN MONEY RESPONSIBLY

Two signatures before the group's money moves

Paying out a member's funeral benefit is one kind of transaction — it goes through the claims process described earlier. But a welfare group also spends money on things that have nothing to do with a member's claim: paying for the AGM venue, buying stationery, moving savings into an investment. Every one of the four real rulebooks studied requires more than one named officer to sign off before money like this leaves the bank account — usually the Chairman, the Treasurer, and sometimes a third person like the Patron.

The app enforces that same rule digitally: an officer can start ("initiate") this kind of payment, but it won't actually go out until the required number of other officers — however many the group decides it wants, not a number baked into the app — have each confirmed it. Every expense, and every donation or grant the group receives, gets logged the same careful way the member dues are: what it was for, which fund it came from or went to, and a receipt attached.

NOT EVERYTHING AT ONCE

What gets built first

Everything explained so far is what the finished platform can do. None of it needs to exist on day one of building it — trying to build all of it at once is how projects like this stall out and never ship anything a real group can use.

The plan: get one real welfare group fully working end to end first — members joining, dues coming in by mobile money, a real claim submitted and paid out correctly — before building anything else. Concretely, that first version includes the core of almost everything already described (joining, contributions, claims, basic reporting, who's-allowed-to-do-what), but leaves out the more advanced pieces for a fast-follow: the AI assistant, the member wallet's actual

top-up screen, the group-spending signatures feature, electing officers digitally, and the full pricing menu all come slightly later, once that first group is genuinely up and running.

THINK OF IT LIKE...

...opening a restaurant with a smaller, well-executed menu before adding the full one. Getting a few dishes completely right and serving real customers teaches you far more than a sprawling menu nobody's actually eaten from yet.

FOLLOWING UP ON THE QUESTIONS YOU ANSWERED

Decisions that have now been made

An earlier version of this guide left four things as open questions for you to weigh in on. Here's where each one landed:

- **How "properly" should the accounting be, on day one? — Decided:** build the core of proper double-entry bookkeeping (Section 12 of the full spec) from the start, but hold off on the fancier accounting features (tracking investments, budgets, petty cash) until a later release. The reasoning: fixing the books to be "properly" correct after the app is already holding real people's money is far riskier than building it right the first time.
- **How should the app charge organisations to use it? — Decided:** a free trial that expires after a set time (not a permanent free tier), then paid plans that combine "how many members" with "which features" — a small single-site group pays less and gets the basics; a large or multi-campus organisation pays more and unlocks more. Payment can be monthly, per-term, or annual, and the group chooses. Taking a cut of every single payment was considered and turned down — it sits too close to the "never touch the welfare money" rule from the previous section.
- **Do we support multi-campus organisations from day one? — Decided:** the underlying design already supports it, but the actual screens for managing several campuses only switch on for organisations paying for the top-tier plan — which a university-sized group would be on anyway, just by virtue of having so many members.

- **How far should a member's wallet go? — Decided:** in only, applied to dues only, as explained above — no cash-out, no sending money to other members.

HONESTY ABOUT WHAT ISN'T DECIDED YET

What's still an open question

A few things genuinely still need a human — probably you — to make a call:

- **Mobile money only at launch, or payroll deductions too?** Three of the four real rulebooks rely on salary deductions, which depend on cooperation from an outside employer's payroll system — that's a dependency the app itself can't control.
- **Should uploading the actual constitution document be a day-one feature, or can groups start with just the settings wizard?**
- **Which countries beyond Ghana are we building for in the first year?** That changes which data-protection and financial rules apply.
- **Should the default number of required signatures for spending money (two signatures? three?) be fixed, or left blank until each group sets its own?** The rulebooks studied lean toward three (Chairman, Treasurer, and a third officer), but that's not universal.
- **Does the member wallet need a lawyer to look at it before launch?** Even with cash-out and transfers switched off, "money sitting in an app" is close enough to financial-services territory that a proper legal review is worth doing rather than relying on this guide's own reasoning.

If you read nothing else in this guide, read this: a document that claims everything is settled is usually hiding its real decisions, not avoiding them. This one tries not to.

QUICK REFERENCE

A short glossary

Tenant

One organisation using the app — its own "apartment" in the building. Not a person; an entire welfare association is one tenant.

Multi-tenancy

The building itself: one piece of software serving many separate organisations, each walled off from the others.

Rule engine

The part of the app that reads an organisation's own settings (its "rules") and applies them, instead of having those rules hardwired into the program.

Rule version / effective date

When a group changes a rule, the old version is kept, not deleted, tagged with the date it stopped applying — so anything that happened under the old rule is judged fairly, by the rule that was actually in force at the time.

Double-entry ledger

An accountant's way of recording money: every movement is written down on two sides that must balance, and nothing already recorded is ever edited — only corrected with a new entry.

Obligation vs. payment

What a member *owed* for a given month, versus what they *actually paid* — kept as two separate facts so a late payment can't quietly be relabelled as on-time.

RBAC (role-based access control)

The wristband system: who's allowed to do what inside the app, decided by each organisation for itself.

Maker-checker / segregation of duties

The rule that whoever starts a financial action shouldn't be the same person who approves it.

Anti-dumping

The set of guardrails that stop someone from ignoring dues for months and then paying everything back right before claiming a benefit.

Good standing

Whether a member is currently up to date enough on their dues to be eligible for benefits, under that organisation's own rules.

Account vs. Membership

Your login (Account) is one thing; your relationship with a particular welfare group (Membership) is another. One Account can hold several Memberships, switchable in-app, without mixing up money or eligibility between groups.

Wallet

A top-up-only balance a member can draw on for their own dues — money can go in and be spent on obligations, but never be withdrawn as cash or sent to another member.

Proxy payment

Paying toward someone else's due directly — the charge hits the payer's phone or card, but the payment settles the named recipient's obligation, with both people's records staying clearly and honestly attributed.

Disbursement Authorization

The digital version of requiring two or three named officers to jointly sign off before the group's money moves for a non-benefit expense (rent, procurement, an investment) — separate from the claims-approval process a member benefit goes through.

Resolution vote

A vote on a proposal rather than a person — "raise the due to GHS 30," for instance — using the same voting mechanism as an officer election.